RAFT LIBRARY

DEVELOPER MANUAL

June 5, 2026

Contents

1 Introduction

This manual describes the Raft library from two developer viewpoints.

The first viewpoint is the application developer who wants to add cluster capability to a domain application. In that role the developer should not need to understand every election rule or replication edge case. The important questions are where application state enters the replicated log, how reads are made safe, how commands and snapshots are encoded, and what the Java and C++ integration surfaces expect.

The second viewpoint is the Raft machinery developer who maintains the library itself. In that role the developer must reason about terms, voting, leader state, log matching, snapshots, read leases, joint consensus, wire compatibility and cross-language parity. The Java and C++ implementations are intended to be functionally equivalent at the Raft boundary even though the language idioms, runtime frameworks and internal structure differ.

The library exists to let applications run in a Raft cluster without embedding domain knowledge in the consensus machinery. Reference-data distribution is an important example: writes are relatively rare and governed, while lookups are high-volume and need to happen close to consumers. The same library shape can also support other master-data or configuration-data applications where a central management path must feed fast replicated lookup state.

1.1 What this manual covers

The manual is organized around the boundary between domain application code and Raft internals:

- Section 3 summarizes the architecture and module layout.
- Section 4 explains how an application uses the library.
- Sections 5 and 6 describe the Java and C++ integration surfaces.
- Section 7 explains the consensus machinery for maintainers.
- Section 8 covers protocol, persistence and snapshots.
- Section 9 explains test strategy and operational checks.

1.2 Terminology

This manual uses *application* for the domain-specific code that owns business state and command encoding. It uses *Raft machinery* for elections, log replication, membership, snapshot metadata, read-safety decisions and transport handling. The boundary is deliberate: applications define what a command means; Raft decides when that command is committed and safe to apply.

2 Design intent

The project should be read as a library project, not as a single reference-data product. Reference data is a useful and important application, but the reusable asset is the cluster capability:

- replicate a deterministic state machine,
- preserve safety across leadership changes,

- expose typed client command and query requests,
- handle cluster membership through replicated configuration entries,
- support Java and C++ applications through equivalent protocol behavior.

The Java implementation is the broader implementation surface. The C++ implementation is developed as a peer implementation and should be brought up to the same behavioral maturity where differences affect Raft semantics. Domain examples may differ temporarily, but the Raft machinery should not.

3 System Overview

The repository contains one shared protocol and two implementation tracks:

Java Maven modules implementing the main Raft runtime, storage, transport, state-machine contracts, example applications, telemetry and tests.
C++ A CMake subtree under `graft-cpp` implementing the same wire protocol and a bounded but growing Raft runtime using Boost.Asio.
Shared wire schema Protobuf messages under `raft-wire/src/main/proto` used by Java and C++.
Jepsen Workload and fault-injection tests under `jepsen`, including mixed Java/C++ scenarios.

3.1 Java module map

Module	Responsibility
`raft-wire`	Protocol model, protobuf mapping and shared wire messages.
`raft-core`	Raft node state machine: terms, votes, log matching, commit, snapshot handling, read safety and membership application.
`raft-membership`	Cluster configuration model and joint-consensus majority calculations.
`raft-state-machine`	Application state-machine interfaces: `SnapshotStateMachine`, `QueryableStateMachine` and result-returning variants.
`raft-storage`	In-memory and file-backed persistent state and log stores.
`raft-transport-netty`	Netty transport client/server for Raft and client RPCs.
`raft-runtime`	Application bootstrap, adapter, operational handlers and policy hooks.
`raft-app-kv`	Basic key-value demo application.
`raft-app-reference`	Reference-data application example and policy behavior.
`raft-telemetry`	Telemetry exporter support.
`raft-dist`	Runnable distribution entry point.
`raft-tests`	Unit, integration and cross-module tests.

3.2 C++ layout

The C++ subtree is intentionally smaller but follows the same conceptual layers:

`include/graft/core` Raft node, application state-machine and snapshot codec.
`include/graft/runtime` Runtime and RPC handler abstractions.
`include/graft/storage` Persistent state store.
`include/graft/transport` Boost.Asio client/server transport.
`src` Implementations of the core, runtime, storage, transport and CLI.
`tests` Catch2 tests for the C++ core and runtime behaviors.

3.3 Boundary principle

The central design rule is:

> Raft owns replication safety. The application owns command meaning.

That has practical consequences:

- Raft persists terms, votes, log entries, commit index, membership and snapshot metadata.
- The application persists its own state only through deterministic command application and application snapshot bytes.
- Cluster configuration is part of Raft state, not part of the domain model, although it is persisted beside the replicated application data path.
- Application code should not decide majorities, refresh read leases, send vote requests or mutate the Raft log directly.

3.4 Shared protocol

Java and C++ interoperate because they share:

- protobuf message definitions,
- envelope framing,
- client command and query request/response shapes,
- membership and join/reconfiguration messages,
- telemetry and cluster-summary messages,
- snapshot install messages.

Cross-language compatibility should be tested at the protocol boundary. If Java and C++ disagree about status strings, read-safety conditions, membership majorities or snapshot metadata, that is a library bug unless the difference is explicitly documented as domain-specific.

4 Application Developer Guide

This section is for developers who want to use the library to give an application Raft cluster behavior.

4.1 What an application must provide

An application supplies four things:

1. A command encoding for writes.
2. A deterministic state machine that applies committed commands.
3. A snapshot encoding for the application state.
4. Optional query handling for local reads after Raft has established read safety.

The application does not implement leader election, quorum calculation, log replication or membership transitions.

4.2 Command design

Commands are opaque byte payloads to Raft. They should be designed as a stable application protocol:

- Use explicit command types rather than positional assumptions.
- Include stable command identifiers when client retries must be idempotent.
- Keep commands deterministic: all replicas must produce the same state from the same ordered log.
- Avoid embedding local time, random values or local host state unless those values are part of the command chosen before replication.
- Treat command formats as versioned public contracts once multiple deployed nodes may run different software versions.

For reference-data style applications, typical commands are `UPSERT_PRODUCT`, `REMOVE_PRODUCT`, `UPSERT_VARIANT` and `REMOVE_VARIANT`. For a key-value application, commands may be `PUT`, `CAS`, `DELETE` and `CLEAR`.

4.3 State-machine contract

The state machine is called only for committed entries. It should therefore be written as pure replicated state mutation, not as a client-facing validation service. Validation that decides whether a write is accepted before entering Raft belongs in admission, authentication or authorization policy. Validation that is part of deterministic domain semantics may happen inside the state machine.

```
1 public interface SnapshotStateMachine {
2     void apply(long term, byte[] command);
3     byte[] snapshot();
4     void restore(byte[] snapshotData);
5 }
6
7 public interface QueryableStateMachine extends SnapshotStateMachine {
8     byte[] query(byte[] request);
```

```
}
```

Listing 1: Java state-machine shape

The C++ interface carries the same idea using C++ types:

```
class ApplicationStateMachine {
public:
    virtual std::string apply(std::int64_t index,
                              std::int64_t term,
                              std::string_view command) = 0;
    virtual std::string snapshot() const = 0;
    virtual std::string query(std::string_view request) const = 0;
    virtual void restore(std::string_view snapshot) = 0;
};
```

Listing 2: C++ state-machine shape

4.4 Snapshots

Application snapshots contain application state only. Raft wraps that payload with Raft metadata, including the cluster configuration at the snapshot boundary. This distinction matters because a snapshot restore must restore both:

- the application's lookup state,
- the Raft membership and log boundary that make the snapshot safe.

Application developers should not store cluster configuration inside their domain snapshot unless the domain itself has a reason to display or audit it. Cluster configuration is Raft machinery state.

4.5 Reads and read leases

Applications may expose read queries through a queryable state machine. The application should treat such reads as local state inspection. It should not try to contact peers or decide whether the local node is a safe reader.

The runtime decides whether a read may be served:

- Followers redirect ordinary linearizable reads to the known leader.
- A leader may serve a read if its quorum-backed read lease is fresh.
- If the lease is stale, the leader must refresh a quorum barrier before serving.
- If the barrier cannot be refreshed, the response should ask the client to retry.

4.6 Client-facing policies

The runtime has policy hooks around client commands:

Admission Should this local node admit the write at all?
Authentication Who is making the request?
Authorization Is the authenticated principal allowed to perform this write?

These hooks are application-facing. They are not Raft safety hooks. A policy may reject a write for domain or security reasons, but it must not claim a command is committed. Only Raft commit and state-machine application can do that.

4.7 Cluster configuration storage

Applications do not need a separate domain table or object for cluster configuration. Configuration is replicated through Raft internal log entries and is captured in Raft snapshot metadata. The library persists it through the same Raft persistence path as terms, votes and log state.

The important application responsibility is operational: the deployment must provide durable storage for the Raft node. If the node loses its Raft storage it may lose its term, vote, log and membership history. That is a cluster operational failure, not a domain state-machine feature.

5 Java Library

The Java implementation is a Maven reactor. Application developers normally interact with the runtime and state-machine modules rather than constructing `RaftNode` directly.

5.1 Application integration path

A Java application should start from:

- `SnapshotStateMachine` or `QueryableStateMachine` in `raft-state-machine`,
- `BasicAdapter` and application module hooks in `raft-runtime`,
- a transport factory, normally Netty through `raft-transport-netty`,
- storage from `raft-storage`.

The normal shape is:

1. Define application command and query payloads.
2. Implement the state machine.
3. Create an adapter or module that supplies that state machine to the runtime.
4. Configure peers, local identity, storage and transport.
5. Let the runtime expose client command/query and operational endpoints.

5.2 State machine interfaces

`SnapshotStateMachine` is the minimal contract. It receives committed commands and can produce/restore application snapshots. `QueryableStateMachine` adds local query support. The result-returning state-machine variant is useful when a committed command should return a typed application result, for example CAS.

State-machine implementations should be thread-conscious. Raft calls are synchronized at the node level where necessary, but application state should still avoid exposing mutable internal collections directly.

5.3 Runtime modules

`RaftApplicationModule` separates application packaging from the shared runtime. It lets an application declare supported runtime modes, CLI commands and an adapter factory. This is where reference-data or other domain packages plug into the generic executable.

`RaftApplicationFactory` creates a runnable adapter for a local peer. It receives:

- local peer identity,
- initially known peers,
- optional join seed,
- runtime configuration,
- timeout settings.

An application should prefer these runtime hooks over direct construction of transport and node internals.

5.4 Policy hooks

The Java runtime exposes:

- `ClientWriteAdmissionPolicy`,
- `ClientCommandAuthenticator`,
- `ClientCommandAuthorizer`.

Use these for domain and security decisions before a command enters Raft. Keep the policy decisions explicit and testable. Do not hide domain rejection inside a state-machine no-op unless that no-op is itself the deterministic domain behavior.

5.5 Storage

The Java storage layer separates persistent term/vote state from the log store. Production deployments should use durable stores. Tests may use the in-memory stores, but those stores are not a recovery mechanism.

Cluster configuration is applied from internal replicated entries and is also carried in snapshot metadata. Application code should treat configuration as Raft-owned state.

5.6 Javadocs and Maven publishing

The reactor is configured to attach source and Javadoc jars. The public API should therefore keep class and method Javadocs useful for application developers. In particular, boundary interfaces should document whether a method is application-owned or Raft-owned.

The usual publishing readiness checks are:

- `mvn -q test`
- `mvn -q -DskipTests package`
- verify source and Javadoc jars under module `target` directories
- ensure publishing metadata and signing are configured before Maven Central upload

6 C++ Library

The C++ implementation lives under `graft-cpp`. It is built separately with CMake so C++ deployments do not depend on the Maven reactor. It shares the protobuf schema and envelope protocol with Java.

6.1 Build model

The C++ build creates:

- `graft_proto`: generated protobuf types,
- `graft_transport`: reusable library target for core/runtime/storage/transport,
- `graft_smoke`: command-line probe and runnable modes,
- `graft_unit_tests`: Catch2 tests when Catch2 v3 is available.

Basic workflow:

```
cmake -S graft-cpp -B graft-cpp/build
cmake --build graft-cpp/build
ctest --test-dir graft-cpp/build --output-on-failure
```

Listing 3: C++ build workflow

The code requires C++20. Older Boost.Asio installations may still reference `std::result_of`; the CMake target defines `BOOST_ASIO_DISABLE_STD_RESULT_OF` to remain compatible with such headers in C++20 mode.

6.2 Application boundary

The C++ application boundary mirrors the Java state-machine boundary:

- commands are opaque bytes or strings at the Raft layer,
- application state machines apply committed commands,
- snapshots are application bytes wrapped by Raft metadata,
- queries are served only after the runtime establishes read safety.

The current C++ example state machine is key-value oriented. Reference-data mode behavior exists at the policy boundary, but the Java implementation remains the fuller reference-data domain implementation.

6.3 Runtime and transport

Boost.Asio provides client and server transport over the shared envelope protocol. The active server modes wire a local C++ `RaftNode` to an RPC handler and transport server. The smoke CLI can also act as a client for Java or C++ peers.

Use C++ smoke commands to validate boundary behavior:

```
graft-cpp/build/graft_smoke client-put 127.0.0.1 11082 k v1 cpp-node
graft-cpp/build/graft_smoke client-get 127.0.0.1 11082 k cpp-node
graft-cpp/build/graft_smoke cluster-summary 127.0.0.1 11082 cpp-cli
```

```
graft-cpp/build/graft_smoke reconfiguration-status 127.0.0.1 11082 cpp-cli
```

Listing 4: Selected C++ probe commands

6.4 Protobuf toolchain

The C++ build uses `protoc` and `libprotobuf`. They should come from the same package manager prefix. A warning such as compiler version `33.x` versus library version `6.33.x` is usually a CMake version-comparison artifact, but mixed prefixes can cause real compile or link problems. Pin the compiler explicitly when needed:

```
cmake -S graft-cpp -B graft-cpp/build \
  -DProtobuf_PROTOC_EXECUTABLE=/opt/local/bin/protoc
```

Listing 5: Pinning protoc

6.5 Parity expectations

C++ should be judged against Java at the Raft behavior boundary:

- status values such as `ACCEPTED`, `REDIRECT`, `RETRY` and `INVALID`,
- vote and append semantics,
- snapshot install and restore semantics,
- read barrier and read-lease behavior,
- joint-consensus majority rules,
- membership admission and finalization,
- mixed Java/C++ cluster behavior.

Differences in domain examples can be tolerated temporarily. Differences in Raft safety semantics should be treated as defects.

7 Raft Internals

This section is for developers changing the Raft machinery itself.

7.1 Core state

A Raft node maintains:

- current term,
- voted-for candidate,
- local role: follower, candidate or leader,
- replicated log,
- commit index and last-applied index,
- known leader,
- cluster membership configuration,
- leader-side replication progress for followers,
- read-lease and heartbeat timing state.

The safety rules are the usual Raft rules: terms are monotonic, a node votes at most once per term, candidates must have sufficiently up-to-date logs, leaders append entries in order, and committed entries must not be lost across leader changes.

7.2 Elections

Election behavior is driven by timeout and vote responses:

1. A follower whose election deadline expires becomes a candidate.
2. The candidate increments term, votes for itself and sends vote requests.
3. A candidate becomes leader only after satisfying the configured majority rule.
4. Any node that observes a higher term steps down.

Membership matters. In stable configuration, election success requires a majority of current voters. In joint consensus, election success requires a majority of current voters and a majority of next voters.

7.3 AppendEntries and commit

Leaders replicate log entries using AppendEntries. Followers reject requests whose previous index/term does not match their log. Leaders backtrack until the matching prefix is found, then append missing entries.

Commit advancement must respect both term and membership:

- The leader should only commit entries known replicated on a required majority.
- Joint consensus requires both current and next majorities.
- Entries from older terms require the normal Raft care: leadership should be established through an entry in the current term before relying on older entries for commitment.

7.4 Read lease and read barrier

Linearizable reads require proof that the leader is still authoritative. The library uses a read-lease/read-barrier model:

- A leader records successful quorum contact through heartbeat/append responses.
- While that evidence is fresh, it may serve local queries.
- If evidence is stale, it must refresh a quorum heartbeat barrier.
- If the barrier fails, the query returns retry/redirect behavior rather than serving stale state.

The same membership rules apply here: joint consensus requires current and next majorities for the barrier to be safe.

7.5 Joint consensus

Membership changes are replicated through internal log entries. A safe transition uses two phases:

1. Enter joint consensus with current and next configurations active.
2. Finalize into the next stable configuration after the joint entry is committed.

During joint consensus, majority calculations must be split, not flattened. A single large majority across the union is not sufficient. The implementation must evaluate current voters and next voters independently.

7.6 Internal commands

Internal Raft commands are log entries whose meaning is Raft-owned rather than domain-owned. Membership changes are the main example. Internal commands must be decoded and applied before or alongside domain state updates so that effective configuration by log index remains correct.

Application developers should not create internal commands directly. Machinery developers should keep internal command encoding versioned and compatible across Java and C++.

8 Wire, Storage and Snapshots

8.1 Wire protocol

All cross-node and client-facing transport uses a shared protobuf schema. The transport envelope carries:

- correlation id,
- term,
- message type,
- protobuf payload.

Java maps protobuf messages through the `ProtoMapper`. C++ generates protobuf classes from the same schema and uses the same envelope framing. When a new message is added, both languages must be updated and mixed smoke coverage should be considered.

8.2 Status vocabulary

Status strings are part of the cross-language contract. Examples include:

`ACCEPTED` A client command was accepted after commit/apply.
`REDIRECT` The contacted node is not the leader and can identify a leader.
`RETRY` The request may be retried, commonly after read-barrier failure.
`INVALID` The request payload or operation is invalid.
`UNAUTHORIZED` Authentication or authorization failed.

Avoid introducing language-specific synonyms. If Java says `ACCEPTED`, C++ should not say `OK` for the same behavior.

8.3 Persistent state

Raft persistence includes:

- current term,
- voted-for candidate,
- known leader where applicable,
- log entries,
- snapshot boundary and metadata,
- cluster configuration.

Application state is persisted indirectly by applying committed entries and by snapshotting application state. If a node restarts, it must reconstruct both Raft metadata and application state consistently.

8.4 Snapshot wrapping

Snapshots have two layers:

Application payload Bytes produced by the state machine.
Raft wrapper Metadata such as last included index, last included term and cluster configuration at that boundary.

This wrapper is essential for restart and catch-up. A follower receiving a snapshot must restore application state and update Raft membership/log boundary state consistently.

8.5 Configuration storage

Cluster configuration is not an external domain object. It is part of the Raft log and snapshot metadata. The application may expose cluster configuration through operational APIs, but it should not duplicate configuration as domain state unless it is doing so for audit or display.

8.6 Protocol evolution

When changing the shared protocol:

1. Update the protobuf schema.
2. Regenerate Java and C++ bindings.
3. Update Java mapping and C++ mapping/handling.
4. Add unit tests in both languages where behavior changes.
5. Add mixed smoke or Jepsen coverage when the change affects cross-language behavior.

Prefer additive schema changes. If a field meaning changes, document the version expectations and test mixed-version behavior before relying on it operationally.

9 Testing and Operations

9.1 Test layers

The project uses several layers of verification:

Java unit tests Core election, append, snapshot, membership, read-lease, adapter and storage behavior.
Java transport tests Netty client/server behavior and response handling.
C++ unit tests C++ core, membership, state-machine, snapshot and RPC behavior.
Mixed smoke tests Java/C++ interoperability at the shared wire boundary.
Jepsen Fault-injection and workload checks for stronger system-level confidence.

Use the smallest test layer that can prove a change, then add broader tests when the change crosses module, language or safety boundaries.

9.2 Java test workflow

```
mvn -q test
mvn -q -DskipJepsenTests=true test
mvn -q -DskipTests package
```

Listing 6: Java test workflow

Unit tests should avoid real Netty resources unless the test is explicitly about transport behavior. Use no-op or in-memory transport fakes for Raft core tests so older platforms with lower file-descriptor limits do not fail while constructing test nodes.

9.3 C++ test workflow

```
cmake -S graft-cpp -B graft-cpp/build
cmake --build graft-cpp/build
ctest --test-dir graft-cpp/build --output-on-failure
```

Listing 7: C++ test workflow

When CMake finds the wrong protobuf tools, pin `protoc`. If Boost.Asio fails around removed standard-library traits, check the C++ compatibility macro described in Section 6.

9.4 Mixed Java/C++ checks

The mixed smoke scripts validate that both implementations agree on the shared protocol:

```
graft-cpp/run-mixed-smoke.sh
graft-cpp/run-mixed-smoke-java-leader.sh
graft-cpp/run-mixed-membership-smoke.sh
graft-cpp/run-mixed-membership-cpp-leader-smoke.sh
graft-cpp/run-mixed-snapshot-smoke.sh
graft-cpp/run-mixed-suite.sh
```

Listing 8: Mixed implementation checks

Run these after changing:

- protobuf messages,
- status strings,
- client command/query behavior,
- membership behavior,
- read-barrier behavior,
- snapshot install or snapshot metadata.

9.5 Jepsen

Jepsen should be used when a change affects safety under concurrency, failure or network partitions. In particular, run or update Jepsen scenarios for:

- leader election changes,
- commit-index changes,
- read-lease/read-barrier changes,
- membership changes,
- mixed Java/C++ cluster behavior,
- client workload semantics such as CAS.

The Jepsen suite is not a replacement for unit tests. It is the system-level check that the library still behaves correctly when timing and partitions are hostile.

9.6 Operational notes

A Raft deployment needs durable per-node storage. Losing local Raft storage can lose votes, terms, log entries and membership state. For application developers, this means the storage path is part of the cluster deployment contract, even if the domain state itself looks like an in-memory hashtable.

Operators should monitor:

- current leader,
- term,
- commit index and last applied index,
- follower replication lag,
- quorum health,
- current and next quorum health during joint consensus,
- stuck reconfiguration age,
- snapshot install events.

10 Appendix: Commands and Checklists

10.1 Repository build commands

Command	Purpose
`mvn-qtest`	Run Java tests in the Maven reactor.
`mvn-q-DskipJepsenTests=truetest`	Run Java tests while skipping Jepsen integration.
`mvn-q-DskipTestspackage`	Build packages, source jars and Javadoc jars.
`cmake-Sgraft-cpp-Bgraft-cpp/build`	Configure the C++ build.
`cmake--buildgraft-cpp/build`	Build C++ targets.
`ctest--test-dirgraft-cpp/build--output-on-failure`	Run C++ tests.
`graft-cpp/run-mixed-suite.sh`	Run mixed Java/C++ smoke checks.

10.2 Application developer checklist

1. Define command and query payload formats.
2. Implement deterministic command application.
3. Implement application snapshot and restore.
4. Decide whether command results are needed.
5. Decide whether local queries are needed.
6. Add admission/authentication/authorization policy where appropriate.
7. Configure durable Raft storage.
8. Test leader, follower redirect, retry and invalid-payload behavior.
9. Test restart from log and snapshot.
10. Test membership changes if the application will operate dynamic clusters.

10.3 Raft machinery developer checklist

1. Identify whether the change affects safety, liveness, protocol or packaging only.
2. Add focused Java and/or C++ unit tests for the affected rule.
3. Check Java/C++ status vocabulary and response shapes.
4. Check stable and joint-consensus majority behavior.
5. Check read-barrier behavior if reads are involved.
6. Check snapshot metadata and restore behavior if compaction or install is involved.
7. Run mixed smoke tests for cross-language protocol changes.
8. Run Jepsen when failures, partitions, membership or linearizable reads are involved.

10.4 Common failure interpretations

Leader rejects write The contacted node may be follower, learner, decommissioned or unable to commit.

Query returns retry The leader could not refresh a quorum-backed read barrier.

C++ Boost.Asio result_of error Older Boost.Asio was compiled as C++20 without the compatibility macro.

Protobuf compiler/library warning CMake found `protoc` and `libprotobuf` with version strings that do not compare cleanly, often because package manager prefixes differ.

Mixed smoke status mismatch Java and C++ disagree at the protocol contract. Treat this as a parity defect unless explicitly domain-specific.